FICHAS DE PRÁCTICAS

UD03 — Fichas de prácticas: prácticas guiadas y retos

Módulo: Sistemas Operativos Monopuesto · 1º SMR
Unidad: UD03 — Configuración de Linux (RA3)

Hay una ficha por cada apartado de los apuntes, y cada ficha tiene dos partes:

- **Parte A — Prácticas guiadas.** Son los cuadros "Vamos a practicar" de los apuntes, paso a paso. Aquí se añade lo que tienes que entregar: una **captura de pantalla** (📸) en los pasos marcados, que demuestre que lo has hecho tú, y la respuesta a cada "Reflexiona". Esta parte es **obligatoria** y se evalúa aunque te atasques en los retos.
- **Parte B — Retos.** Situaciones reales sin pasos: decides tú qué hacer. En la mayoría, **algo está roto** en tu VM y tu trabajo es averiguar qué, arreglarlo y demostrar que funciona. También hay algún ejercicio en papel.

Puedes (y debes) consultar tus apuntes, las páginas de manual (`man comando`) y la ayuda de cada comando (`comando --help`). Lo que no vale es aplicar soluciones al azar hasta que algo funcione: un técnico que arregla algo sin saber por qué se ha roto no ha terminado su trabajo.

Cómo se evalúan

La nota de UD03 se reparte así: **fichas de prácticas, 35%** y **examen, 65%**. Dentro del 35% de las fichas:

Parte	Peso en la nota de la unidad	Qué se valora
Parte A — Prácticas guiadas	15%	Que esté hecha y completa: todas las capturas 📸, con tu *prompt*, y las respuestas a los "Reflexiona".
Parte B — Retos	20%	Tus informes y entregas. En los retos 🔧 se valora **cómo has trabajado** (síntoma, pruebas reales, razonamiento), **aunque no llegues a resolverlo**.

Las fichas son, sobre todo, una herramienta para aprender: el examen pregunta por lo mismo que practicas en ellas.

Antes de empezar

Cómo entregar las capturas

- En cada captura de la terminal debe verse el ***prompt***, con tu usuario y el nombre de tu equipo (por ejemplo, `alumno@apellido-ud02-linux:~$`). Una captura en la que no se vea quién ha ejecutado el comando, y dónde, no vale como evidencia.

- Nombra cada captura con su código: la marcada como 📸 **G1.1-a** se guarda como `G1.1-a.png`.
- Entrega todo en un único documento por ficha, en el orden de la ficha: capturas, respuestas a los "Reflexiona" e informes de los retos.

Tipos de ejercicio

- 🧭 **Práctica guiada:** en tu VM Ubuntu, siguiendo los pasos.
- 💻 **Reto en la VM:** en tu VM, sin pasos.
- 🔧 **Reto de avería:** antes de empezar, se rompe algo en tu VM sin que sepas qué.
- 📝 **Ejercicio en papel:** no necesitas el equipo.

Cómo se prepara un reto de avería 🔧

Tu profesor o profesora te dará el archivo **`saboteoControlado.sh`**. Cópialo a tu carpeta personal de la VM (por ejemplo, a través de la carpeta compartida de UD02). Para cada reto 🔧:

1. Toma una instantánea de tu VM con el nombre `Antes del reto X.Y`.
2. Ejecuta `sudo bash saboteoControlado.sh X.Y` (por ejemplo, `sudo bash saboteoControlado.sh 3.2`). El script te pedirá confirmar que has tomado la instantánea, aplicará la avería sin decirte qué ha tocado y, si hace falta, reiniciará la VM.
3. Resuelve el reto siguiendo el método de diagnóstico.
4. Cuando creas que está resuelto, ejecuta `sudo bash saboteoControlado.sh --comprobar X.Y` y haz una captura del resultado. Esa captura forma parte de la entrega.

No abras el script para ver qué hace: el reto consiste precisamente en averiguarlo. Si tu profesor o profesora organiza los retos **por parejas**, en lugar del script te dará una tarjeta de avería para que la apliques en la VM de tu compañero o compañera, sin decirle qué has tocado. Si eres tú quien prepara la avería, toma primero la instantánea en su VM.

Método de diagnóstico

Ante cualquier avería, sigue siempre este orden. No te saltes pasos, aunque creas que ya sabes qué pasa:

1. **Síntoma:** ¿qué ves exactamente? Anota el mensaje literal, no tu interpretación.
2. **Hipótesis:** ¿qué podría causar eso? Piensa en al menos dos causas posibles.
3. **Prueba:** ¿qué comando o comprobación te permite confirmar o descartar cada hipótesis?
4. **Interpretación:** ¿qué te dice el resultado de la prueba?
5. **Solución:** aplica el cambio mínimo necesario.
6. **Comprobación:** demuestra que el problema ha desaparecido, y que no has roto otra cosa.
7. **Prevención:** ¿cómo se podría haber evitado?

El informe de incidencia

Cada reto 🔧 se entrega con un informe como este, con las capturas de las pruebas que hayas hecho:

Apartado	Qué tienes que escribir
Síntoma	Lo que observaste, con el mensaje de error literal.
Hipótesis	Las causas posibles que consideraste.
Pruebas	Cada comando que ejecutaste, qué esperabas ver y qué viste.
Causa	Qué estaba mal, exactamente.
Solución	Qué cambiaste y por qué ese cambio lo arregla.
Comprobación	Cómo demuestras que ya funciona, incluida la captura de `--comprobar`.
Prevención	Cómo evitar que vuelva a pasar.

Un informe con el apartado "Pruebas" vacío o de una sola línea no está completo, aunque el problema esté resuelto.

Regla de las evidencias: cada comando que cites en tu informe tiene que aparecer en una captura de **tu** VM, con su *prompt* y su resultado. Un informe que explica comandos que no aparecen en tus capturas no cuenta como hecho.

Para qué sirven estas fichas: para aprender, no para demostrar que ya sabes. Equivocarte en un reto, probar algo que no funciona y anotarlo en tu informe es exactamente lo que se espera: así es como se aprende a diagnosticar. Si le pides la solución a una IA, entregarás un informe, pero llegarás al examen sin haber practicado lo que se va a preguntar.

Las pistas de comandos

Cada reto incluye una línea 🔎 con **algunos** comandos que te pueden servir. No están en orden, no llevan las opciones que necesitas y puede que no los uses todos: te dicen por dónde buscar, no qué escribir. Busca cada uno en tus apuntes o en su página de manual (`man comando`) antes de ejecutarlo.

Ficha 1 — Arranque y sesiones

Apartado relacionado: 1 · **CE:** a

Parte A — Prácticas guiadas

G1.1 🧭 — Cambia de target y consulta el registro de arranque

1. `systemctl get-default` — anota el resultado (debería ser `graphical.target`). 📸 **G1.1-a**
2. `sudo systemctl isolate multi-user.target` — la pantalla debería quedarse en modo texto.
3. Inicia sesión con tu usuario y contraseña en esa terminal de texto, y ejecuta `whoami` para comprobar que sigues siendo el mismo usuario que en el escritorio. 📸 **G1.1-b** (haz una foto

con el móvil o usa **Ver** → **Tomar captura de pantalla** en VirtualBox, porque la terminal de texto no tiene herramienta de capturas)

4. `sudo systemctl isolate graphical.target` — recuperas el entorno gráfico, sin haber reiniciado en ningún momento.
5. `journalctl -b | less` — despliega el registro del arranque actual (usa las flechas para moverte, `q` para salir) y localiza alguna línea relacionada con el montaje del sistema de archivos raíz. 📸 **G1.1-c** con esa línea visible.
6. `loginctl list-sessions` — anota el ID de tu sesión.
7. `loginctl show-session <ID>`, sustituyendo `<ID>` por el que acabas de anotar. 📸 **G1.1-d** con las dos órdenes y su resultado.

Reflexiona (respuesta escrita): explica, con tus propias palabras, qué diferencia real hay a nivel de systemd entre `multi-user.target` y `graphical.target`. No te quedes en "uno tiene interfaz gráfica y el otro no": explica la relación de dependencia entre ambos targets.

Entrega de la parte A: capturas G1.1-a a G1.1-d y la respuesta a la reflexión.

Parte B — Retos

Reto 1.1 🔧 — El equipo que arranca sin escritorio

Preparación: `sudo bash saboteoControlado.sh 1.1`

Situación: una compañera de la oficina te llama: "Esta mañana he encendido el ordenador y me sale una pantalla negra con letras pidiendo usuario y contraseña. Ayer funcionaba perfectamente. No he tocado nada".

Tu misión: averigua por qué el sistema arranca así, deja el equipo arrancando de nuevo en modo gráfico **de forma permanente** y demuéstralo reiniciando.

🔎 **Comandos que te pueden servir** (búscalos en tus apuntes o en `man` para ver cómo se usan): `systemctl get-default` · `systemctl set-default` · `journalctl -b`

Para pensar mientras trabajas: ¿habría bastado con un comando `isolate`? ¿Qué habría pasado en el siguiente arranque?

Comprobación: `sudo bash saboteoControlado.sh --comprobar 1.1` 📸

Reto 1.2 💻 — ¿Quién más está conectado?

Preparación (la haces tú, no hay avería): cambia a la terminal virtual 3 pulsando `Tecla anfitrión + F3` (en VirtualBox, la tecla anfitrión es el `Ctrl` derecho) e inicia sesión con tu usuario. Vuelve al escritorio con `Tecla anfitrión + F2` (o `F1`, según la versión). Esa sesión se queda abierta.

Situación: al revisar un equipo, encuentras una sesión abierta que nadie está usando. Por seguridad, las sesiones abandonadas deben cerrarse.

Tu misión:

1. Averigua cuántas sesiones hay abiertas ahora mismo, de qué usuario es cada una y en qué terminal está.

2. Identifica cuál es la tuya, la que estás usando en este momento, y explica **cómo lo sabes**.
3. Cierra la otra sesión **sin reiniciar el equipo y sin cerrar la tuya**. El comando que necesitas no aparece en los apuntes: búscalo en `man loginctl`.
4. Demuestra que ya solo queda tu sesión. 📸

🔎 **Comandos que te pueden servir** (búscalos en tus apuntes o en `man` para ver cómo se usan): `loginctl list-sessions` · `loginctl show-session` · `echo $XDG_SESSION_ID` · `man loginctl`

Ejercicio 1.3 📝 — ¿Qué harías en cada caso?

Para cada situación, indica qué acción es la adecuada (cerrar sesión, bloquear, apagar, reiniciar, cambiar de target con `isolate` o cambiar el target por defecto con `set-default`) y el comando o la opción del menú que usarías. Justifica cada respuesta en una línea.

1. Te levantas cinco minutos de tu puesto para ir a por un café y dejas documentos abiertos.
2. Acabas de instalar una actualización del núcleo de Linux.
3. Un servidor del departamento no necesita escritorio nunca, y quieres que deje de cargarlo en cada arranque para ahorrar memoria.
4. Necesitas comprobar, solo durante unos minutos, cómo se ve un equipo en modo texto, y después seguir trabajando en el escritorio.
5. Terminas tu turno y otro compañero va a usar el mismo equipo con su propia cuenta.
6. Te vas de vacaciones dos semanas y nadie va a usar el equipo.

Ejemplo resuelto: *1. Bloquear.* Los documentos siguen abiertos exactamente como los dejé, pero nadie puede verlos ni tocarlos sin mi contraseña. En el menú del escritorio, "Bloquear", o con el atajo `Super+L`.

Ficha 2 — Interfaces de usuario y entornos de escritorio

Apartado relacionado: 2 · **CE:** b, c

Parte A — Prácticas guiadas

G2.1 🧭 — Instala y elige un segundo entorno de escritorio

1. `sudo apt update` — deja el índice de paquetes al día antes de instalar nada.
2. `sudo apt install xubuntu-desktop` (o el meta-paquete que se te indique: `kubuntu-desktop` para KDE Plasma, `ubuntu-mate-desktop` para MATE). Tardará varios minutos. 📸 **G2.1-a** con el final de la instalación.
3. Cuando termine, cierra la sesión desde el menú del escritorio (**no apagues la VM**).
4. En la pantalla de inicio de sesión, pulsa el icono de engranaje junto al campo de contraseña y comprueba que aparecen las dos sesiones disponibles. 📸 **G2.1-b** con la lista de sesiones desplegada (usa **Ver** → **Tomar captura de pantalla** en VirtualBox).
5. Entra con el nuevo entorno de escritorio. 📸 **G2.1-c** del nuevo escritorio.

G2.2 🧭 — Personaliza y compara

1. En tu entorno original (por ejemplo, GNOME), abre "Ajustes" y cambia el fondo de pantalla y el tema. 📸 **G2.2-a** antes y 📸 **G2.2-b** después.
2. Cierra sesión, entra con el segundo entorno y localiza su panel de configuración equivalente (por ejemplo, "Configuración del sistema" en KDE Plasma, o el "Gestor de configuración" en XFCE).
3. Cambia también ahí el fondo de pantalla y el tema. 📸 **G2.2-c** antes y 📸 **G2.2-d** después.
4. Abre el monitor de recursos (o el gestor de tareas gráfico) en cada sesión y anota cuánta RAM usa el sistema recién iniciado en cada una. 📸 **G2.2-e** y 📸 **G2.2-f**, una por entorno.
5. Completa una tabla con: RAM en reposo, primera impresión visual, y dónde está la configuración de personalización en cada entorno.

Reflexiona (respuesta escrita): si desinstalaras ahora mismo el entorno que no estás usando, ¿qué riesgo real correrías? ¿Por qué es más seguro simplemente no usarlo que desinstalarlo?

Entrega de la parte A: capturas G2.1-a a G2.2-f, la tabla comparativa y la respuesta a la reflexión.

Parte B — Retos

Reto 2.1 💻 — Línea de comandos o entorno gráfico: cronómetro en mano

Situación: en el departamento hay una discusión: unos dicen que la terminal "es más rápida" y otros que "es de hace 40 años". Te piden datos, no opiniones.

Tu misión: haz cada una de estas tareas **dos veces**, una desde el entorno gráfico y otra desde la terminal, y cronometra cuánto tardas en cada caso:

1. Crear 20 carpetas llamadas `tema01`, `tema02`... hasta `tema20` dentro de tu carpeta personal. Pista para la terminal: investiga la "expansión de llaves" de bash.
2. Averiguar qué versión del núcleo de Linux tiene tu sistema.
3. Cambiar el fondo de pantalla por otra imagen.
4. Averiguar cuánto espacio libre queda en el disco.

Borra las carpetas entre un intento y otro, para que las condiciones sean iguales.

🔎 **Comandos que te pueden servir** (búscalos en tus apuntes o en `man` para ver cómo se usan): `mkdir` · `uname` · `df` · `gsettings`

Entrega: una tabla con los tiempos, una captura de la terminal con los comandos que has usado, y una conclusión de 5-6 líneas que responda: ¿en qué tipo de tareas gana cada interfaz, y por qué? ¿Cuál de las dos usarías para administrar un equipo que está en otro edificio?

Reto 2.2 💻 — Un escritorio para el aula de equipos antiguos

Situación: un instituto va a reutilizar un aula con ordenadores de hace diez años: 2 GB de RAM y un procesador de dos núcleos. Te piden que decidas qué entorno de escritorio instalar en ellos.

Tu misión:

1. Toma una instantánea y ajusta la configuración de tu VM en VirtualBox para que tenga 2 GB de RAM y 2 núcleos de CPU, como los equipos del aula.
2. Mide, **con datos reales de tu VM**, cuánta RAM usa el sistema recién iniciado con al menos dos entornos de escritorio. Explica cómo has hecho la medición para que sea justa: mismo momento tras el inicio de sesión y ninguna aplicación abierta.
3. Recomienda un entorno de escritorio para el aula y justifica la decisión con tus medidas. Ten en cuenta que los usuarios serán alumnado que viene de Windows.
4. Deja tu VM iniciando sesión con el entorno elegido y personalízalo para que resulte familiar a un usuario de Windows: barra de tareas abajo y menú de aplicaciones en la esquina.
5. Al terminar, devuelve la RAM y la CPU de tu VM a sus valores originales.

🔎 **Comandos que te pueden servir** (búscalos en tus apuntes o en `man` para ver cómo se usan): `free -h` · *Monitor del sistema* · *Configuración de la VM en VirtualBox → Sistema*

Entrega: tabla de medidas con sus capturas, recomendación justificada y captura del escritorio personalizado.

Reto 2.3 🔧 — La sesión que ha desaparecido

Preparación: `sudo bash saboteoControlado.sh 2.3` (necesitas haber hecho antes la práctica G2.1).

Situación: un usuario usaba a diario el segundo entorno de escritorio que tenía instalado. Hoy, en la pantalla de inicio de sesión, solo le aparece GNOME. Dice que "alguien ha estado limpiando programas".

Tu misión: averigua qué ha pasado, recupera la sesión perdida sin reinstalar el sistema y demuéstralo iniciando sesión con ella.

🔎 **Comandos que te pueden servir** (búscalos en tus apuntes o en `man` para ver cómo se usan): `ls /usr/share/xsessions` · `dpkg -l` · `/var/log/apt/history.log` · `apt install`

Pista (léela solo si llevas 15 minutos sin avanzar): la lista de sesiones de la pantalla de login no es mágica: cada sesión la aporta un paquete instalado. ¿Qué comando de los apuntes te dice qué pasó con un paquete? Y APT guarda un historial de todo lo que instala y desinstala: búscalo en `/var/log/apt/`.

Comprobación: `sudo bash saboteoControlado.sh --comprobar 2.3` 💣

Ejercicio 2.4 📝 — ¿Terminal o entorno gráfico?

Para cada situación, di qué interfaz elegirías y por qué:

1. Hay que renombrar 500 fotos para que empiecen por la fecha.
2. Un usuario que nunca ha usado Linux tiene que cambiar su contraseña.
3. Tienes que revisar un servidor que está en un centro de datos a 300 km.
4. Quieres descubrir qué opciones de accesibilidad ofrece el sistema.
5. Hay que repetir la misma configuración en los 30 equipos de un aula.

Ficha 3 — Sistemas de archivos y recuperación

Apartado relacionado: 3 · **CE:** d, e

Parte A — Prácticas guiadas

G3.1 🧭 — Monta un "disco" de prueba a mano

1. `sudo dd if=/dev/zero of=/root/disco_prueba.img bs=1M count=100` — crea un archivo de 100 MB que va a hacer de disco.
2. `sudo mkfs.ext4 /root/disco_prueba.img` — lo formatea con ext4.
3. `sudo mkdir /mnt/prueba` — crea el punto de montaje.
4. `sudo mount -o loop /root/disco_prueba.img /mnt/prueba` — lo monta manualmente.
5. `df -h | grep prueba` — comprueba que aparece montado. 📸 **G3.1-a**
6. `echo "hola" | sudo tee /mnt/prueba/saludo.txt` — crea un archivo dentro.
7. `sudo umount /mnt/prueba` — desmóntalo.
8. `ls /mnt/prueba` — comprueba que la carpeta aparece vacía. 📸 **G3.1-b** con los pasos 6 a 8.

Reflexiona (respuesta escrita): el archivo `saludo.txt` no se ha borrado: sigue existiendo dentro de `disco_prueba.img`. Entonces, ¿por qué "desaparece" de `/mnt/prueba` en cuanto desmontas? ¿Qué te dice esto sobre lo que significa realmente "montar" algo?

G3.2 🧭 — Añade un segundo disco y haz que se monte solo

1. **Con la VM apagada**, en VirtualBox abre **Configuración** → **Almacenamiento**, selecciona el controlador SATA y pulsa el icono de **añadir disco duro**. Elige **Crear**, tipo VDI, reservado dinámicamente, de **1 GB**, con el nombre `datos`. 📸 **G3.2-a** de la configuración de almacenamiento con los dos discos.
2. Arranca la VM y ejecuta `lsblk`. Localiza el disco nuevo de 1G, normalmente `sdb`.
3. `sudo parted /dev/sdb --script mklabel gpt mkpart datos ext4 0% 100%` — **comprueba antes que `sdb` es el disco nuevo, el de 1G**.
4. `lsblk` de nuevo: ahora aparece `sdb1`. 📸 **G3.2-b**
5. `sudo mkfs.ext4 /dev/sdb1`
6. `sudo mkdir /mnt/datos`
7. `sudo blkid /dev/sdb1` — anota el UUID de la **partición**.
8. `sudo nano /etc/fstab` y añade al final `UUID=<tu-uuid> /mnt/datos ext4 defaults 0 2`. 📸 **G3.2-c** con la línea añadida (`cat /etc/fstab`).
9. `sudo systemctl daemon-reload`
10. `sudo mount -a` y `df -h /mnt/datos`. 📸 **G3.2-d**
11. Reinicia la VM y comprueba de nuevo con `df -h /mnt/datos` que sigue montado. 📸 **G3.2-e**

Reflexiona (respuesta escrita): ¿por qué se ejecuta `mount -a` antes de reiniciar? ¿Qué ganas probándolo así?

G3.3 🧭 — Localiza tu propio swap

`swapon --show`, `free -h` y `cat /etc/fstab`: comprueba cuánto swap tiene tu sistema, de dónde procede y cuál es su línea en `fstab`. 📸 **G3.3-a**

Reflexiona (respuesta escrita): ¿por qué esta línea no corre el riesgo que evitamos con el UUID (que `/dev/sdX` pueda cambiar de nombre entre arranques)?

G3.4 🧭 — Rompe y repara `fstab`, de forma controlada

1. Toma una instantánea de tu VM con el nombre "Antes de romper fstab". 📸 **G3.4-a** de la lista de instantáneas.
2. `sudo nano /etc/fstab` y, en la línea de `/mnt/datos`, cambia un par de caracteres del UUID.
3. Reinicia la VM. El arranque parecerá bloqueado durante un minuto y medio: systemd está esperando a un disco con ese UUID.
4. Lee el mensaje que aparece y anótalo literalmente. 📸 **G3.4-b** (**Ver** → **Tomar captura de pantalla** en VirtualBox).
5. Si el mensaje dice que la cuenta de root está bloqueada, reinicia la VM desde el menú de VirtualBox (**Máquina** → **Reiniciar**).
6. Al empezar el arranque, mantén pulsada `Mayús` (arranque BIOS) o pulsa `Esc` varias veces (arranque UEFI) hasta que aparezca el menú de GRUB.
7. Con la entrada de Ubuntu seleccionada, pulsa `e`, añade al final de la línea que empieza por `linux` un espacio y `init=/bin/bash`, y arranca con `Ctrl+X` o `F10`. 📸 **G3.4-c** de la línea editada, antes de arrancar.
8. `mount -o remount,rw /`
9. `blkid /dev/sdb1` para consultar el UUID correcto, y `nano /etc/fstab` para corregir la línea. 📸 **G3.4-d**
10. `sync` y después `reboot -f`.
11. Comprueba que el sistema arranca con normalidad y que `df -h /mnt/datos` vuelve a mostrar el disco montado. 📸 **G3.4-e**

Ojo con el teclado: en el editor de GRUB, y puede que también en la terminal que arranca con `init=/bin/bash`, el teclado funciona con la distribución estadounidense. Con un teclado español, la `/` está en la tecla `-`, el `=` en la tecla `¡` y el `-` en la tecla `'`.

Reflexiona (respuesta escrita): explica en 3-4 líneas por qué esta incidencia no tiene nada que ver con el journaling de ext4 que estudiaste en UD01. Y después: si hubieras escrito la línea con `defaults,nofail`, ¿qué habría pasado en el paso 3?

Entrega de la parte A: capturas G3.1-a a G3.4-e y las respuestas a las cuatro reflexiones.

Parte B — Retos

Reto 3.1 💻 — Un disco más para el departamento

Situación: el departamento de administración necesita otro disco más para sus datos, separado del sistema y del que añadiste en la práctica G3.2, que esté siempre disponible en la ruta `/datos` al encender el equipo. Esta vez no hay pasos: ya sabes hacerlo.

Tu misión:

1. Con la VM **apagada**, añade en VirtualBox otro disco virtual de 1 GB.
2. Arranca la VM y localiza el disco nuevo. ¿Qué nombre de dispositivo le ha dado Linux? ¿Por qué no es el mismo que el del disco de la práctica G3.2?
3. Crea en él una partición y dale formato ext4. Puedes usar la aplicación gráfica "Discos" o la terminal: tú eliges, pero explica por qué.
4. Haz que se monte **automáticamente** en `/datos` en cada arranque, identificándolo de la forma más fiable.
5. Decide qué opciones de montaje pones. Investiga qué hace la opción `nofail` y decide si la usas o no. Justifícalo: ¿qué pasaría en el arranque si algún día el disco no estuviera conectado?
6. Antes de reiniciar, comprueba que tu `fstab` no tiene errores. Investiga también qué hace `sudo findmnt --verify`.
7. Reinicia y demuestra que `/datos` está montado sin que hayas hecho nada.
8. Crea en `/datos` una carpeta `documentos` con dos o tres archivos: los vas a necesitar en el reto siguiente.

🔎 **Comandos que te pueden servir** (búscalos en tus apuntes o en `man` para ver cómo se usan): `lsblk` · `parted` · *la aplicación Discos* · `mkfs.ext4` · `blkid` · `nano /etc/fstab` · `systemctl daemon-reload` · `mount -a` · `findmnt --verify`

Entrega: capturas de cada paso, la línea exacta que has añadido a `/etc/fstab` explicando cada uno de sus seis campos, y tu justificación sobre `nofail`.

> Este disco lo vas a necesitar en los retos 3.2, 3.3 y en el reto final: no lo borres.

Reto 3.2 🔧 — Los datos han desaparecido

Preparación: `sudo bash saboteoControlado.sh 3.2` (necesitas haber terminado el reto 3.1).

Situación: el departamento de administración te llama alarmado: "¡La carpeta `/datos` está vacía! Ayer estaba llena de documentos".

Tu misión: averigua dónde están los datos. Antes de tocar nada, responde: ¿se han borrado de verdad? Después, arréglalo para que vuelvan a aparecer en `/datos` y demuestra que el arreglo sobrevive a un reinicio.

🔎 **Comandos que te pueden servir** (búscalos en tus apuntes o en `man` para ver cómo se usan): `ls` · `df -h` · `lsblk -f` · `findmnt` · `nano /etc/fstab` · `umount`

Pista (léela solo si llevas 15 minutos sin avanzar): recuerda lo que aprendiste en UD01 sobre Linux y los nombres de archivo.

Comprobación: `sudo bash saboteoControlado.sh --comprobar 3.2` 💣

Reto 3.3 🔧 — El equipo que no termina de arrancar

Preparación: `sudo bash saboteoControlado.sh 3.3` (necesitas haber terminado el reto 3.1).

Situación: tras "un cambio sin importancia" que hizo un compañero, el equipo ya no llega al escritorio. Se queda en una pantalla de texto con un mensaje de error.

Tu misión:

1. Lee con atención el mensaje y anótalo literalmente.
2. Consigue entrar en el sistema para corregirlo. Ya lo hiciste en la práctica G3.4 con `init=/bin/bash`; esta vez, además, **explica en el informe por qué funciona** esa vía. Si prefieres, puedes usar la otra vía que se menciona en los apuntes: la ISO de Ubuntu en modo *live*.
3. Encuentra el error y corrígelo. Esta vez no es un UUID.
4. Demuestra que el sistema vuelve a arrancar con normalidad.

🔎 **Comandos que te pueden servir** (búscalos en tus apuntes o en `man` para ver cómo se usan): `journalctl -xb` · *menú de GRUB (tecla e)* · `init=/bin/bash` · `mount -o remount,rw /` · `nano /etc/fstab`

Para pensar: ¿habría evitado esta situación la opción `nofail` que investigaste en el reto 3.1? ¿Y el comando `findmnt --verify`? ¿Por qué no es buena idea poner `nofail` en todas las líneas de `fstab`, incluida la del sistema raíz?

Comprobación: `sudo bash saboteoControlado.sh --comprobar 3.3` 💣

Reto 3.4 🔧 — Ampliación: GRUB sin menú

Este reto es opcional. Pide permiso a tu profesor o profesora antes de empezar.

Preparación: `sudo bash saboteoControlado.sh 3.4`

Situación: después de un corte de luz durante una actualización, el equipo arranca y muestra solo esto:

```
grub>
```

Tu misión: desde esa línea de comandos de GRUB, averigua en qué partición está tu sistema, arráncalo a mano y, una vez dentro, regenera la configuración de GRUB para que el problema no se repita. Investiga los comandos `ls`, `set root`, `linux`, `initrd` y `boot` de GRUB.

🔎 **Comandos que te pueden servir** (búscalos en tus apuntes o en `man` para ver cómo se usan): `ls` · `set root` · `linux` · `initrd` · `boot` · `update-grub`

Comprobación: `sudo bash saboteoControlado.sh --comprobar 3.4` 📸, tras reiniciar y ver que el menú de GRUB funciona.

Ejercicio 3.5 📝 — Lee este `fstab`

Estas son las líneas de un archivo `/etc/fstab`:

```
UUID=3f9a1c2e-7b4d-4e8a-9c1f-2a6b8d0e4f11  /          ext4  errors=remount-ro  0  1
/swapfile                                   none       swap  sw                 0  0
UUID=8d2e5b7a-1c3f-4a9e-b6d0-7e4c2f1a9b35  /datos     ext4  defaults,nofail    0  2
/dev/sdc1                                   /copias    ext4  defaults           0  2
UUID=c4a7e9f2-6b1d-4f3a-8e5c-9d2b7a1f6e08  /proyectos ext4  defautls           0  2
```

1. ¿Qué se monta en `/` y qué sistema de archivos tiene?
2. ¿Por qué la segunda línea no usa UUID? ¿Supone algún riesgo?
3. Si un día el disco de `/datos` no está conectado, ¿qué pasará en el arranque? ¿Y si el que falta es el de `/copias`?
4. La cuarta línea funciona hoy, pero es una bomba de relojería. ¿Por qué? ¿Cómo la mejorarías?
5. La quinta línea tiene un error. ¿Cuál es y qué provocará en el próximo arranque?
6. ¿Qué significan el último número de la primera línea (`1`) y el de la tercera (`2`)?

Ficha 4 — Gestión de software

Apartado relacionado: 4 · **CE:** f, g

Parte A — Prácticas guiadas

G4.1 🧭 — Actualiza el índice e instala tu primer paquete

1. `sudo apt update` — refresca el índice de paquetes disponibles. 📸 **G4.1-a**
2. `sudo apt install tree`
3. `tree ~` — pruébalo sobre tu carpeta personal. 📸 **G4.1-b**
4. `apt show tree` — versión, tamaño, descripción y origen del paquete. 📸 **G4.1-c**

G4.2 🧭 — Instala aplicaciones reales con APT y con Snap

1. `sudo apt install refind` — instala rEFInd desde el repositorio oficial.
2. `dpkg -l | grep refind` — comprueba que aparece como `ii`. 📸 **G4.2-a**
3. `sudo add-apt-repository ppa:danielrichter2007/grub-customizer` y `sudo apt update` — añade el PPA de GRUB Customizer.

4. `sudo apt install grub-customizer`. **Si el PPA no tiene paquetes para tu versión de Ubuntu, apt te lo dirá con un error: documéntalo tal cual y sigue.** 📸 **G4.2-b** del resultado, sea cual sea.
5. `sudo snap install vlc`
6. `snap list vlc` 📸 **G4.2-c**
7. `sudo apt remove refind` y `dpkg -l | grep refind` (verás `rc`); después `sudo apt purge refind` y comprueba que no queda ninguna traza. 📸 **G4.2-d** con los dos `dpkg -l`.
8. `sudo snap remove --purge vlc` 📸 **G4.2-e**

Reflexiona (respuesta escrita): ¿por qué crees que una herramienta como GRUB Customizer o rEFInd, que necesitan tocar directamente el gestor de arranque, casi nunca se distribuyen como Snap?

G4.3 🧭 — Provoca (y arregla) unas dependencias rotas

1. Copia a tu VM el archivo `paquete-practica.deb` que te facilitará tu profesor o profesora.
2. `sudo dpkg -i paquete-practica.deb` — la instalación fallará indicando que falta una dependencia. 📸 **G4.3-a** con el mensaje.
3. `sudo apt --fix-broken install` 📸 **G4.3-b**
4. `dpkg -l | grep paquete-practica` — comprueba que ahora aparece como `ii`. 📸 **G4.3-c**
5. Si la dependencia resuelta es `cowsay`, pruébala: `cowsay "¡Ya funciona!"`.

Reflexiona (respuesta escrita): ¿por qué `dpkg -i` por sí solo no ha sido capaz de resolver la dependencia que faltaba, mientras que `apt` sí lo consigue?

Entrega de la parte A: capturas G4.1-a a G4.3-c y las respuestas a las dos reflexiones.

Parte B — Retos

Reto 4.1 🔧 — `apt update` da errores

Preparación: `sudo bash saboteoControlado.sh 4.1`

Situación: un usuario te dice que el sistema le avisa de que "hay un problema con las actualizaciones". Al ejecutar `sudo apt update`, aparecen errores.

Tu misión: interpreta el error, localiza qué lo provoca, soluciónalo **sin reinstalar nada** y demuestra que `apt update` vuelve a terminar sin errores.

🔎 **Comandos que te pueden servir** (búscalos en tus apuntes o en `man` para ver cómo se usan): `apt update` · `grep -r` · `/etc/apt/sources.list.d/` · `mv`

Para pensar: ¿es un problema de tu conexión a Internet o de la configuración del sistema? ¿Cómo lo has distinguido?

Comprobación: `sudo bash saboteoControlado.sh --comprobar 4.1` 📸

Reto 4.2 🔧 — "No me deja instalar nada"

Preparación: `sudo bash saboteoControlado.sh 4.2`

Situación: vas a instalar `cowsay` y `apt` no avanza: se queda esperando, o se niega, con un mensaje que habla de un "bloqueo" (*lock*). Un compañero te dice: "Eso se arregla borrando el archivo de bloqueo, lo he visto en un foro".

Tu misión:

1. Lee el mensaje de error completo. Contiene una pista muy concreta: ¿cuál?
2. Averigua qué proceso está provocando el bloqueo y qué está haciendo. Te ayudará el comando `ps -p <número> -o pid,user,tty,etime,cmd`.
3. Decide si es seguro detenerlo y, si lo es, hazlo. Resuelve la situación **sin borrar ningún archivo del sistema y sin reiniciar**.
4. Instala `cowsay`.

🔎 **Comandos que te pueden servir** (búscalos en tus apuntes o en `man` para ver cómo se usan): `ps -p` · `kill`

Para pensar: ¿por qué existe ese bloqueo? ¿Qué podría pasar si borras el archivo de bloqueo mientras otro proceso está instalando paquetes? ¿Por qué en este caso era seguro detener el proceso, y en qué caso no lo sería?

Comprobación: `sudo bash saboteoControlado.sh --comprobar 4.2` 💣

Reto 4.3 🔧 — El programa que no se deja desinstalar

Preparación: `sudo bash saboteoControlado.sh 4.3` (tarda un par de minutos y necesita Internet).

Situación: un usuario te escribe: "He intentado desinstalar un programa multimedia con `sudo apt remove` y me dice que no está instalado, pero sigue apareciendo en el menú de aplicaciones y se abre perfectamente".

Tu misión: averigua cuál es el programa y por qué `apt` dice que no está instalado. Desinstálalo por completo, sin dejar datos guardados, y comprueba que ha desaparecido del menú.

🔎 **Comandos que te pueden servir** (búscalos en tus apuntes o en `man` para ver cómo se usan): `apt remove` · `dpkg -l` · `snap list` · `snap remove --purge`

Comprobación: `sudo bash saboteoControlado.sh --comprobar 4.3` 💣

Ejercicio 4.4 📝 — Lee la salida de `dpkg -l`

```
ii  cowsay          3.03+dfsg2-8   all    configurable talking cow
rc  refind         0.14.0.2-1     amd64  boot manager for EFI-based computers
ii  tree           2.1.1-2        amd64  displays an indented directory tree
iF  paquete-prueba 1.0            all    paquete de prueba
ii  libfuse2t64    2.9.9-8.1      amd64  Filesystem in Userspace (library)
```

1. ¿Qué significa cada uno de los estados que aparecen (`ii`, `rc`, `iF`)?
2. ¿Qué comando ejecutaron sobre `refind` para que quede en ese estado? ¿Qué comando lo eliminaría del todo?

3. `paquete-prueba` está a medio configurar. ¿Qué pudo pasar al instalarlo y cómo lo resolverías?
4. Nadie recuerda haber instalado `libfuse2t64`. Antes de desinstalarla, ¿qué comprobarías y por qué? ¿Qué comando usarías para limpiar dependencias que ya no necesita nadie?
5. Escribe el comando equivalente en una distribución Fedora para: actualizar el índice, instalar `cowsay` e instalar un paquete descargado a mano.

Ficha 5 — Asistentes de configuración: red y dispositivos

Apartado relacionado: 5 · **CE:** h

Parte A — Prácticas guiadas

G5.1 🧭 — Configura una IP estática desde el escritorio

1. Abre **Configuración** → **Red**, pulsa el engranaje de tu conexión y entra en la pestaña **IPv4**.
2. Cambia el método de "Automático (DHCP)" a "Manual" e introduce una dirección IP, máscara y puerta de enlace válidas para el modo de red de tu VM (visto en UD02). Aplica los cambios. 📸 **G5.1-a** de la configuración.
3. `ip addr show` — comprueba que se ha aplicado. 📸 **G5.1-b**
4. Desde tu **VM Windows 11**, abre `cmd` y ejecuta `ping <la IP que acabas de fijar>`. 📸 **G5.1-c**
5. Vuelve a **Configuración** → **Red** → **IPv4** y deja el método en "Automático (DHCP)".

Reflexiona (respuesta escrita): para que el `ping` del paso 4 funcione, las dos VMs tienen que "verse" en red. ¿Qué modo de red de VirtualBox es imprescindible para esto, y por qué el NAT simple no serviría?

G5.2 🧭 — La misma IP estática, ahora desde la terminal

1. `nmcli connection show` — anota el nombre exacto de tu conexión.
2. `nmcli connection modify "<conexión>" ipv4.method manual ipv4.addresses "<IP>/<prefijo>" ipv4.gateway "<puerta-de-enlace>" ipv4.dns "8.8.8.8"`
3. `nmcli connection up "<conexión>"` 📸 **G5.2-a** con los pasos 1 a 3.
4. `ip addr show` — comprueba que coincide con la IP que fijaste antes. 📸 **G5.2-b**
5. Repite el `ping` desde tu VM Windows 11. 📸 **G5.2-c**
6. `cat /etc/netplan/*.yaml` — localiza la línea que indica quién gestiona realmente tu red. 📸 **G5.2-d**

Deja la conexión con esta IP estática: la necesitarás en el reto 5.2.

Reflexiona (respuesta escrita): ¿por qué el archivo de Netplan que acabas de ver no contiene tu IP estática? ¿Dónde está realmente guardada esa configuración?

G5.3 🧭 — Explora el asistente de impresoras

Abre el gestor de impresoras (búscalo como "Impresoras") e inicia el asistente para añadir una nueva, aunque no haya ninguna impresora real. 📸 **G5.3-a**, **G5.3-b**... una captura por cada pantalla del asistente, hasta el punto en el que se detiene.

Reflexiona (respuesta escrita): ¿qué información te pide el asistente antes incluso de buscar una impresora? ¿En qué se parece este proceso al de `nmtui`?

Entrega de la parte A: capturas G5.1-a a G5.3 y las respuestas a las tres reflexiones.

Parte B — Retos

Reto 5.1 🔧 — "Tengo red pero no tengo Internet"

Preparación: `sudo bash saboteoControlado.sh 5.1`

Situación: un usuario dice: "El icono de red dice que estoy conectado, pero no se abre ninguna página web".

Tu misión: diagnostica el problema **por capas**, de lo más cercano a lo más lejano, y documenta el resultado de cada comprobación:

1. ¿Tiene el equipo una dirección IP? ¿Es coherente con la red de tu VM?
2. ¿Llega a la puerta de enlace?
3. ¿Llega a una dirección IP de Internet, por ejemplo `8.8.8.8`?
4. ¿Llega a un nombre de dominio, por ejemplo `www.ubuntu.com`?

En cuanto una capa falle, ya sabes dónde buscar. Corrige el problema con el asistente gráfico **o** con `nmcli`, explica por qué has elegido uno u otro, y demuestra que se navega con normalidad.

🔎 **Comandos que te pueden servir** (búscalos en tus apuntes o en `man` para ver cómo se usan): `ip addr` · `ip route` · `ping` · `resolvectl status` · `nmcli connection modify` · *Configuración → Red* · `nmcli connection up`

> Este método de diagnóstico por capas lo vas a volver a usar en el módulo de Redes Locales: no es exclusivo de Linux.

Comprobación: `sudo bash saboteoControlado.sh --comprobar 5.1` 📸

Reto 5.2 🔧 — La IP que llega a todo menos a Internet

Preparación: `sudo bash saboteoControlado.sh 5.2` (tu conexión tiene que tener IP estática, como la dejaste en la práctica G5.2).

Situación: a un equipo con IP estática le han "retocado" la configuración de red. Hace `ping` al equipo de al lado, pero no sale a Internet, ni siquiera por IP.

Tu misión: aplica el mismo diagnóstico por capas que en el reto 5.1. Esta vez te será útil el comando `ip route`. Encuentra el error y corrígelo **desde la terminal, con `nmcli`**, sin volver a DHCP: el equipo debe quedarse con IP estática.

🔎 **Comandos que te pueden servir** (búscalos en tus apuntes o en `man` para ver cómo se usan): `ip route` · `ping` · `nmcli connection show` · `nmcli connection modify` · `nmcli connection up`

Para pensar: ¿qué dato de la configuración de red de tu VM necesitas conocer para poder corregirlo, y dónde lo has encontrado?

Comprobación: `sudo bash saboteoControlado.sh --comprobar 5.2` 📸

Ejercicio 5.3 📝 — Interpreta la configuración de red

Un técnico ha ejecutado estos comandos en un equipo:

```
$ ip addr show enp0s3
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
    link/ether 08:00:27:3a:5c:91 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.47/24 brd 192.168.1.255 scope global dynamic noprefixroute enp0s3
       valid_lft 85912sec preferred_lft 85912sec

$ ip route
default via 192.168.1.1 dev enp0s3 proto dhcp metric 100
192.168.1.0/24 dev enp0s3 proto kernel scope link src 192.168.1.47 metric 100
```

1. ¿Qué dirección IP tiene el equipo? ¿Y qué máscara, escrita en formato decimal (`255.x.x.x`)?
2. ¿Cuál es su puerta de enlace?
3. ¿La IP es estática o la ha recibido por DHCP? Señala las **dos** pistas de la salida que te lo indican.
4. ¿Qué dirección MAC tiene la tarjeta? ¿Qué te dice su principio (`08:00:27`) sobre el tipo de equipo? Pista: investiga a qué fabricante corresponde ese prefijo.
5. Si quisieras fijar esta misma IP como estática con `nmcli`, ¿qué valores pondrías en `ipv4.addresses` e `ipv4.gateway`?

Ficha 6 — Automatización de tareas

Apartado relacionado: 6 · **CE:** i

Parte A — Prácticas guiadas

G6.1 🧭 — Tu primera tarea programada con `cron`

1. `nano registro.sh` y escribe dentro (cambiando `tu_usuario` por el tuyo):

```
#!/bin/bash
date >> /home/tu_usuario/registro.log
```

2. `chmod +x registro.sh`

3. `./registro.sh` y `cat registro.log` — comprueba que ha añadido la fecha y la hora.
4. `crontab -e` y añade `* * * * * /home/tu_usuario/registro.sh`. 📸 **G6.1-a** con `crontab -l`.
5. Espera 2-3 minutos y ejecuta `cat /home/tu_usuario/registro.log`: deberías ver una línea nueva por minuto. 📸 **G6.1-b**
6. `crontab -e` de nuevo y elimina (o comenta con `#`) esa línea.

G6.2 🧭 — Tres tareas con temporalidades distintas

1. Copia el script `backup_personal.sh` que te facilitará tu profesor o profesora a tu carpeta personal, dale permisos de ejecución (`chmod +x`) y pruébalo una vez a mano.
2. `crontab -e` y añade la línea de copia en semanas alternas de los apuntes, con tu ruta y **escapando el `%`**: `0 3 * * 0 [ $(( $(date +\%V) \% 2 )) -eq 0 ] && /home/tu_usuario/backup_personal.sh`
3. Crea un script `saludo.sh` (cambiando `tu_usuario` por el tuyo):

```
#!/bin/bash
echo "¡Hola $(whoami)! Son las $(date +%H:%M) del $(date +%A)." >>
/home/tu_usuario/saludo.log
```

4. Dale permisos de ejecución y prográmalo para los días laborables: `0 9 * * 1-5 /home/tu_usuario/saludo.sh`.
5. `crontab -l` 📸 **G6.2-a** con las líneas del backup y del saludo.

Reflexiona (respuesta escrita): esa línea de backup se ejecuta técnicamente **todos** los domingos. Entonces, ¿dónde está exactamente la lógica que hace que la copia real solo se haga la mitad de esas veces?

G6.3 🧭 — Programa un apagado automático

1. `sudo crontab -e` — abre el crontab de **root**, no el tuyo.
2. Añade una línea con una hora de dentro de 1-2 minutos: `<minuto> <hora> * * * /usr/sbin/shutdown -h now`. **Guarda antes cualquier trabajo abierto: la VM se va a apagar de verdad.** 📸 **G6.3-a** con `sudo crontab -l`.
3. Espera y comprueba que la VM se apaga sola. Vuelve a encenderla.
4. `sudo crontab -e` y cambia la hora a la definitiva: `55 14 * * * /usr/sbin/shutdown -h now`.
5. `sudo crontab -l` 📸 **G6.3-b**

Reflexiona (respuesta escrita): si hubieras añadido esa misma línea en tu propio `crontab -e` en vez de en el de root, ¿qué habría pasado a la hora programada?

G6.4 🧭 — Una tarea puntual con `at`

1. `echo "echo Tarea completada > /home/tu_usuario/aviso.txt" | at now + 5 minutes`

2. `atq` — comprueba que la tarea está en la cola. 📸 **G6.4-a**
3. Espera los 5 minutos y ejecuta `cat /home/tu_usuario/aviso.txt`. 📸 **G6.4-b**

Reflexiona (respuesta escrita): si el script de la práctica G6.1 hubiera usado `>> registro.log` en vez de una ruta absoluta, ¿dónde habría acabado escribiendo cuando lo lanzara `cron`, y por qué no sería donde tú esperabas?

Entrega de la parte A: capturas G6.1-a a G6.4-b y las respuestas a las tres reflexiones.

Parte B — Retos

Reto 6.1 🔧 — La copia de seguridad que nunca se hizo

Preparación: `sudo bash saboteoControlado.sh 6.1`

Situación: un usuario tiene programada una copia de seguridad cada dos minutos, para probarla. "El crontab está bien, lo he mirado mil veces, pero no se crea ninguna copia".

Tu misión:

1. Comprueba si `cron` está ejecutando la tarea o no. Investiga cómo ver en el registro del sistema lo que hace `cron`: prueba `journalctl -u cron`.
2. Si la ejecuta, averigua por qué no hace nada. Tienes una pista: `cron` no te enseña los errores en pantalla. ¿Cómo podrías conseguir que los guarde en un archivo para poder leerlos?
3. Corrige el problema, **en la línea del crontab y dentro del script**, y demuestra que la copia se crea.
4. Cuando funcione, cambia la tarea a una hora razonable para que no siga ejecutándose cada dos minutos.

🔎 **Comandos que te pueden servir** (búscalos en tus apuntes o en `man` para ver cómo se usan): `journalctl -u cron` · `crontab -l` · `crontab -e` · `>> archivo.log 2>&1`

Comprobación: `sudo bash saboteoControlado.sh --comprobar 6.1` 📸 (antes del paso 4, cuando ya se haya creado alguna copia).

Reto 6.2 💻 — Tareas de mantenimiento para un aula

Situación: te encargan automatizar el mantenimiento de los equipos de un aula. Programa en tu VM:

1. Que cada día laborable, a las 8:05, se guarde en `/home/<tu_usuario>/mantenimiento.log` la fecha, la hora y el espacio libre del disco. Tendrás que escribir tú el script.
2. Que el equipo se apague solo a las 15:00 de lunes a viernes, pero **no** los fines de semana.
3. Que el día 1 de cada mes, a las 7:30, se ejecute `apt update` y el resultado quede guardado en un archivo de registro.
4. Una tarea que se ejecute **una única vez**, mañana a las 10:00, y que cree un archivo recordatorio en tu carpeta personal.

Decide en qué crontab va cada tarea (el tuyo o el de root) y justifícalo. Para cada una, explica cómo has comprobado que funciona **sin esperar** a la hora real, con capturas.

🔎 **Comandos que te pueden servir** (búscalos en tus apuntes o en `man` para ver cómo se usan): `crontab -e` · `sudo crontab -e` · `at` · `atq` · `df -h` · `date`

Ejercicio 6.3 📝 — Lee y escribe líneas de `crontab`

A. Explica con palabras cuándo se ejecuta cada línea:

```
30 7 * * 1-5   /home/ana/copia.sh
*/15 * * * *   /home/ana/comprobar.sh
0 0 1 1 *      /home/ana/felicitar.sh
0 22 * * 0,6   /home/ana/limpieza.sh
```

B. Escribe la línea de `crontab` para:

1. Cada día a las 23:45.
2. Cada 10 minutos, pero solo entre las 8:00 y las 14:59.
3. Los lunes y los jueves a las 12:00.
4. El día 15 de cada mes a medianoche.

C. Cada una de estas líneas tiene un error que hará que no funcione como su autor espera. Encuéntralo y corrígelo:

1. `0 3 * * * backup.sh` (en el crontab de un usuario normal)
2. `0 9 * * 1-5 echo "Hoy es $(date +%A)" >> /home/luis/dia.log`
3. `0 14 * * * /usr/sbin/shutdown -h now` (en el crontab de `luis`, no en el de root)
4. `60 8 * * * /home/luis/tarea.sh`

Ejemplo resuelto (A, primera línea): se ejecuta a las 7:30 de lunes a viernes, es decir, cada día laborable antes de empezar la jornada.

Reto final 🔧 — El equipo de Laura

Preparación: `sudo bash saboteoControlado.sh final` (necesitas haber terminado el reto 3.1).

Situación: llega un parte de incidencias, tal cual lo ha escrito la usuaria:

> "Mi ordenador está fatal. Al encenderlo ya no sale el escritorio, sale una pantalla negra con letras. Cuando consigo entrar, no puedo abrir ninguna página web. Y encima la carpeta de datos del departamento aparece vacía. Necesito que funcione hoy, por favor."

Tu VM tiene **varias averías a la vez**. No sabes cuántas.

Tu misión: deja el equipo funcionando por completo y entrega un **informe de incidencia profesional**, dirigido a la usuaria y a tu responsable, que incluya:

1. Un apartado técnico, con el método de diagnóstico completo para **cada** avería: síntoma, hipótesis, pruebas, causa, solución y comprobación.
2. Un resumen de tres o cuatro líneas, sin tecnicismos, para la usuaria: qué pasaba y qué se ha hecho.
3. Recomendaciones de prevención.
4. La captura de `sudo bash saboteoControlado.sh --comprobar final`.

🔎 **Comandos que te pueden servir** (búscalos en tus apuntes o en `man` para ver cómo se usan): *todos los que has usado en las fichas 1 a 6: los síntomas del parte te dicen por qué ficha empezar*

Tiempo orientativo: dos sesiones.

Relación con RA y CE

Ficha	Parte A (guiada)	Parte B (retos)	CE
1	G1.1	1.1, 1.2, 1.3	a
2	G2.1, G2.2	2.1, 2.2, 2.3, 2.4	b, c
3	G3.1 a G3.4	3.1 a 3.5	d, e
4	G4.1 a G4.3	4.1 a 4.4	f, g
5	G5.1 a G5.3	5.1 a 5.3	h
6	G6.1 a G6.4	6.1 a 6.3	i
Reto final	—	Integrador	a, d, e, h